

[< Return to Classroom](#)

Data Modeling with Postgres

REVIEW

CODE REVIEW 10

HISTORY

Meets Specifications

Great job, you are ready to go! 🎉 Clearly, you have acquired all the important concepts from this project. Wish you all the best for the upcoming projects! 🙌

Tip: If you are interested in knowing better how to analyze the Postgres performance, you are encouraged to read [this article](#). Also, [this article](#) is about the pros/cons analysis of the Postgres

Table Creation

The script, `create_tables.py`, runs in the terminal without errors. The script successfully connects to the Sparkify database, drops any tables if they exist, and creates the tables.

Good job! You successfully drop all existing tables and create all necessary tables 🙌

CREATE statements in `sql_queries.py` specify all columns for each of the five tables with the right data types and conditions.

Perfect! You specify all columns for each of the five tables with the right data types and conditions. 🙌

ETL

The script, `etl.py`, runs in the terminal without errors. The script connects to the Sparkify database, extracts and processes the `log_data` and `song_data`, and loads data into the five tables.

Since this is a subset of the much larger dataset, the solution dataset will only have 1 row with values for value containing ID for both `songid` and `artistid` in the fact table. Those are the only 2 values that the query in the `sql_queries.py` will return that are not-NONE. The rest of the rows will have NONE values for those two variables.

Well done! You correctly implement all required functions in `etl.py` 🎉

INSERT statements are correctly written for each table, and handle existing records where appropriate. `songs` and `artists` tables are used to retrieve the correct information for the `songplays` INSERT.

Good job! Your INSERT statements are correctly written for each table

Code Quality

The README file includes a summary of the project, how to run the Python scripts, and an explanation of the files in the repository. Comments are used effectively and each function has a docstring.

Rate this review

START

Great summary and clearly explain the main purpose of this project 🍑
Tip: For making it better, you are encouraged to give some examples of how the dimension table helps to reduce the duplication. It will be nice to see an example of how the duplication occurred (maybe just referencing the variable/column name) and which tables. Also, I would also encourage you to use images, like screenshots of your final tables, and adding more details about your project, such as the dataset, how you cleaned the data.

Scripts have an intuitive, easy-to-follow structure with code separated into logical functions. Naming for variables and functions follows the PEP8 style guidelines.

The code is clean and neat! 🍑 And required docstrings are added for each function!

 [DOWNLOAD PROJECT](#)

10

[CODE REVIEW COMMENTS](#)



[RETURN TO PATH](#)

[Return to Classroom](#)

Data Modeling with Postgres

REVIEW

CODE REVIEW 10

HISTORY

▼ sql_queries.py 8

```
1 # DROP TABLES
2
3 songplay_table_drop = "DROP TABLE IF EXISTS songplays;"
4 user_table_drop = "DROP TABLE IF EXISTS users;"
5 song_table_drop = "DROP TABLE IF EXISTS songs;"
6 artist_table_drop = "DROP TABLE IF EXISTS artists;"
7 time_table_drop = "DROP TABLE IF EXISTS time;"
```

AWESOME

Good job! You successfully drop all existing tables 🍏

```
8
9 # CREATE TABLES
10
11 songplay_table_create = ("""
12 CREATE TABLE IF NOT EXISTS songplays
13 ( songplay_id serial \
14 ,start_time timestamp NOT NULL REFERENCES time(start_time) \
15 ,user_id int NOT NULL REFERENCES users(user_id) \
```

AWESOME

You correctly add Not Null condition on both `user_id` and `start_time`. 🍏

```
16 ,level varchar \
17 ,song_id varchar REFERENCES songs(song_id) \
18 ,artist_id varchar REFERENCES artists(artist_id) \
19 ,session_id int NOT NULL \
20 ,location varchar \
21 ,user_agent varchar \
22 ,PRIMARY KEY (songplay_id) \
```

AWESOME

Good job! You correctly set PRIMARY KEY on songplay_id 100

```
23 ,UNIQUE (artist_id, song_id, user_id, session_id, start_time) \
24 ); \
25 """
26
27 user_table_create = (""" \
28 CREATE TABLE IF NOT EXISTS users \
29 ( user_id int \
30 ,first_name varchar \
31 ,last_name varchar \
32 ,gender varchar \
33 ,level varchar \
34 ,PRIMARY KEY (user_id) \
```

AWESOME

Good job! You correctly set PRIMARY KEY on user_id 100

Rate this review

START

```

35 ); \
36 """)
37
38 song_table_create = ("" \
39 CREATE TABLE IF NOT EXISTS songs \
40 ( song_id   varchar \
41  ,title     varchar \
42  ,artist_id varchar REFERENCES artists(artist_id) \
43  ,year      smallint \
44  ,duration  numeric \
45  ,PRIMARY KEY (song_id) \

```



AWESOME

Good job! You correctly set PRIMARY KEY on song_id 100

```

46 ); \
47 """)
48
49 artist_table_create = ("" \
50 CREATE TABLE IF NOT EXISTS artists \
51 ( artist_id varchar \
52  ,name      varchar \
53  ,location  varchar \
54  ,latitude  numeric \
55  ,longitude numeric \
56  ,PRIMARY KEY (artist_id) \

```



AWESOME

Good job! You correctly set PRIMARY KEY on artist_id 100

```

57 ); \
58 """)
59
60 time_table_create = ("" \
61 CREATE TABLE IF NOT EXISTS time \
62 ( start_time timestamp \
63  ,hour      int \
64  ,day       int \
65  ,week      int \
66  ,month     int \
67  ,year      int \
68  ,weekday   int \
69  ,PRIMARY KEY (start_time) \

```



AWESOME

Good job! You correctly set PRIMARY KEY on start_time 100

```

70 ); \
71 """)
72
73 # INSERT RECORDS
74
75 songplay_table_insert = ("" \
76 INSERT INTO songplays \
77 ( start_time \
78  ,user_id \
79  ,level \
80  ,song_id \
81  ,artist_id \
82  ,session_id \
83  ,location \
84  ,user_agent \
85  ) \
86 VALUES(%s, %s, %s, %s, %s, %s, %s, %s); \
87 """)
88
89 user_table_insert = ("" \
90 INSERT INTO users \
91 ( user_id \
92  ,first_name \
93  ,last_name \
94  ,gender \
95  ,level \
96  ) \
97 VALUES(%s, %s, %s, %s, %s) \
98 ON CONFLICT (user_id) \
99 DO UPDATE \
100 SET level = excluded.level; \

```



Good job setting the right CONFLICT constraint. ¹⁰⁰

```

101 """
102
103 song_table_insert = (""" \
104 INSERT INTO songs \
105   ( song_id \
106     ,title \
107     ,artist_id \
108     ,year \
109     ,duration \
110   ) \
111 VALUES (%s, %s, %s, %s, %s) \
112 ON CONFLICT (song_id) \
113 DO NOTHING; \
114 """
115
116 artist_table_insert = (""" \
117 INSERT INTO artists \
118   ( artist_id \
119     ,name \
120     ,location \
121     ,latitude \
122     ,longitude \
123   ) \
124 VALUES (%s, %s, %s, %s, %s) \
125 ON CONFLICT (artist_id) \
126 DO NOTHING; \
127 """
128
129 time_table_insert = (""" \
130 INSERT INTO time \
131   ( start_time \
132     ,hour \
133     ,day \
134     ,week \
135     ,month \
136     ,year \
137     ,weekday \
138   ) \
139 VALUES(%s, %s, %s, %s, %s, %s, %s) \
140 ON CONFLICT (start_time ) \
141 DO NOTHING; \
142 """
143
144 # FIND SONGS
145
146 song_select = (""" \
147 SELECT songs.song_id, artists.artist_id \
148 FROM songs JOIN artists ON songs.artist_id = artists.artist_id \
149 WHERE songs.title = %s AND artists.name = %s AND songs.duration = %s; \
150 """
151
152 # QUERY LISTS
153
154 create_table_queries = [user_table_create, artist_table_create, song_table_create
155 drop_table_queries = [songplay_table_drop, user_table_drop, song_table_drop, arti

```

► etl.py 1

► README.md 1

► create_tables.py

[Return to Classroom](#)

Data Modeling with Postgres

REVIEW

CODE REVIEW 10

HISTORY

► sql_queries.py 8

▼ etl.py 1

```
1 import os
2 import glob
3 import psycopg2
4 import pandas as pd
5 from sql_queries import *
6
7
8 def process_song_file(cur, filepath):
9     """Receives data file containing the song metadata and loads into songs and a
10
11     Keyword arguments:
12     cur -- the connected cursor
13     filepath -- the file name and directory
14     """
15
16     # open song file
17     df = pd.read_json(filepath, lines=True)
18
19     # insert artist record
20     artist_data = df.loc[0, ['artist_id', 'artist_name', 'artist_location', 'artist_l
21     cur.execute(artist_table_insert, artist_data)
22
23     # insert song record
24     song_data = df.loc[0, ['song_id', 'title', 'artist_id', 'year', 'duration']].value
25     song_data[3] = int(song_data[3])
26     cur.execute(song_table_insert, song_data)
27
28
29 def process_log_file(cur, filepath):
30     """Receives a data file containing the logs from user activity and loads into
31
32     Keyword arguments:
33     cur -- the connected cursor
34     filepath -- the file name and directory
35     """
36
37     # open log file
38     df = pd.read_json(filepath, lines=True)
39
40     # filter by NextSong action
41     df = df[df.page == 'NextSong']
42
43     # convert timestamp column to datetime
44     t = df['ts'].astype('datetime64[ms]')
45
46     # insert time data records
47     time_data = (t, t.dt.hour, t.dt.day, t.dt.week, t.dt.month, t.dt.year, t.dt.w
48     column_labels = ('start_time', 'hour', 'day', 'week', 'month', 'year', 'weekd
49     time_df = pd.DataFrame(dict(zip(column_labels, time_data)))
50
51     for i, row in time_df.iterrows():
52         cur.execute(time_table_insert, list(row))
53
54     # load user table
55     user_df = df[['userId', 'firstName', 'lastName', 'gender', 'level']]
56
57     # insert user records
58     for i, row in user_df.iterrows():
59         cur.execute(user_table_insert, row)
```

```

59
60
61 # insert songplay records
62 for index, row in df.iterrows():
63
64     # get songid and artistid from song and artist tables
65     cur.execute(song_select, (row.song, row.artist, row.length))
66     results = cur.fetchone()
67
68     if results:
69         songid, artistid = results
70     else:
71         songid, artistid = None, None
72
73     # insert songplay record
74     songplay_data = (pd.to_datetime(row.ts, unit='ms'), row.userId, row.level,
75                      songid, artistid, row.sessionId, row.location, row.userAgent)
76
77
78 def process_data(cur, conn, filepath, func):
79     """Get all the source files contained in the filepath and execute process_song_data
80
81     Keyword arguments:
82     cur -- the connected cursor
83     conn -- connection string to Postgres server

```

Rate this review

START

AWESOME

Good job adding the required docstring!

```

84     filepath -- the file directory
85     func -- function that will read and insert records to the tables
86     """
87
88     # get all files matching extension from directory
89     all_files = []
90     for root, dirs, files in os.walk(filepath):
91         files = glob.glob(os.path.join(root, '*.json'))
92         for f in files:
93             all_files.append(os.path.abspath(f))
94
95     # get total number of files found
96     num_files = len(all_files)
97     print('{} files found in {}'.format(num_files, filepath))
98
99     # iterate over files and process
100    for i, datafile in enumerate(all_files, 1):
101        func(cur, datafile)
102        conn.commit()
103        print('{} files processed.'.format(i, num_files))
104
105
106 def main():
107     """This program will read all files from song and log directories and properly
108     process them"""
109
110     conn = psycopg2.connect("host=127.0.0.1 dbname=sparkifydb user=student password=student")
111     cur = conn.cursor()
112
113     process_data(cur, conn, filepath='data/song_data', func=process_song_data)
114     process_data(cur, conn, filepath='data/log_data', func=process_log_data)
115
116     conn.close()
117
118
119 if __name__ == "__main__":
120     main()

```

► README.md 1

► create_tables.py

RETURN TO PATH

[Return to Classroom](#)

Data Modeling with Postgres

REVIEW

CODE REVIEW 10

HISTORY

▸ sql_queries.py 8

▸ etl.py 1

▼ README.md 1

```
1 # Data Modeling with Postgres: Song Library
2 > Tim B
3
4 This is an ETL pipeline for a startup **Sparkify**, who wants to analyze the data they've been collecting
5
6 The program creates a tables normalized and modeled in star-schema from two directories of JSON logs on u
```

AWESOME

Great summary and clearly explain the main purpose of this project👍

```
7
8 The goal of this project is to apply, document, and demonstrate technical skills learned on data modeling
9
10 ### Schema
11 Fact and dimension tables were defined for a star schema with an analytic focus.
12
13 #### Fact Table
14 **songplays** - records in log data associated with song plays i.e. records with page NextSong
15 *songplay_id, start_time, user_id, level, song_id, artist_id, session_id, location, user_agent*
16
17 #### Dimension Tables
18 **users** - users in the app
19 *user_id, first_name, last_name, gender, level*
20
21 **songs** - songs in music database
22 *song_id, title, artist_id, year, duration*
23
24 **artists** - artists in music database
25 *artist_id, name, location, lattitude, longitude*
26
27 **time** - timestamps of records in songplays broken down into specific units
28 *start_time, hour, day, week, month, year, weekday*
29
30 ### Database creation script
31 create_tables.py - A script for creating and recreating the target database is provided for easy editions
32
33 ### Database creation notebook
34 test.ipynb - A Jupiter notebook to validate and test the successful creation of empty target tables before
35
36 ### ETL Script
37 etl.py - An ETL script automatically loops through the logs and songs directories, transforms the data us
38
39 ### ETL Notebook
40 etl.ipynb - A Jupiter notebook is provided to document and demonstrate the ETL pipeline, step by step.
41
42 > Note: The data directories include just a sub-set of the files.
43 ##### /data/log_data/
44 ##### /data/song_data/
45
```

▸ create_tables.py

[Return to Classroom](#)

Data Modeling with Postgres

REVIEW

CODE REVIEW 10

HISTORY

▸ sql_queries.py 8

▸ etl.py 1

▸ README.md 1

▼ create_tables.py

```
1 import psycopg2
2 from sql_queries import create_table_queries, drop_table_queries
3
4
5 def create_database():
6     """
7     - Creates and connects to the sparkifydb
8     - Returns the connection and cursor to sparkifydb
9     """
10
11     # connect to default database
12     conn = psycopg2.connect("host=127.0.0.1 dbname=studentdb user=student password=student")
13     conn.set_session(autocommit=True)
14     cur = conn.cursor()
15
16     # create sparkify database with UTF8 encoding
17     cur.execute("DROP DATABASE IF EXISTS sparkifydb")
18     cur.execute("CREATE DATABASE sparkifydb WITH ENCODING 'utf8' TEMPLATE template0")
19
20     # close connection to default database
21     conn.close()
22
23     # connect to sparkify database
24     conn = psycopg2.connect("host=127.0.0.1 dbname=sparkifydb user=student password=student")
25     cur = conn.cursor()
26
27     return cur, conn
28
29
30 def drop_tables(cur, conn):
31     """
32     Drops each table using the queries in `drop_table_queries` list.
33     """
34     for query in drop_table_queries:
35         cur.execute(query)
36         conn.commit()
37
38
39 def create_tables(cur, conn):
40     """
41     Creates each table using the queries in `create_table_queries` list.
42     """
43     for query in create_table_queries:
44         cur.execute(query)
45         conn.commit()
46
47
48 def main():
49     """
50     - Drops (if exists) and Creates the sparkify database.
51
52     - Establishes connection with the sparkify database and gets
53     cursor to it.
54
55     - Drops all the tables.
56
57     - Creates all tables needed.
58
59     - Finally, closes the connection.
60     """
61     cur, conn = create_database()
62
63     drop_tables(cur, conn)
64     create_tables(cur, conn)
65
66     conn.close()
67
68
69 if __name__ == "__main__":
70     main()
```

RETURN TO PATH

Rate this review

START

[Return to Classroom](#)

Data Modeling with Postgres

REVIEW	CODE REVIEW 10	HISTORY
Review #3 (this review) Reviewed 16h ago STUDENT NOTES etl.py file updated only.		
This review Reviewed 16h ago Review #2 Reviewed 18h ago Review #1 Reviewed 2 days ago		

RETURN TO PATH

Rate this review